JourneySync

Real-Time Cross-Channel Identity Resolution & Customer Journey Stitching Platform

Round 1 Submission – Idea Proposal

Track: Cross-Channel Journey Stitching – Building a Unified Customer Journey Intelligence Platform

Codestreet 2026 - Team Cinephiles

July 27, 2026

Contents

1 Solution Overview **2**
1.1 Problem Statement Selected . . . 2
1.2 Proposed Solution . . . 2
1.3 Expected Business & Societal Impact . . . 2
1.4 Success Metrics . . . 3
1.5 Implementation Approach . . . 3

2 Technical Details **4**
2.1 Technology Stack & Frameworks . . . 4
2.2 Architecture Diagram . . . 4
2.3 Event Resolution Logic . . . 5
2.4 Scalability Considerations . . . 5

1 Solution Overview

1.1 Problem Statement Selected

Cross-Channel Journey Stitching: Building a Unified Customer Journey Intelligence Platform.

Customers interact with financial institutions across a wide range of channels – mobile apps, websites, call centers, chatbots, email, SMS, and physical branches. Although these interactions belong to a single customer journey, they are typically stored in disconnected systems with little or no communication between them. This fragmentation makes it difficult for a bank to answer a deceptively simple question: *"What actually happened to this customer, across every channel, in the last 24 hours?"*

The consequences are tangible: customers are forced to repeat themselves across channels, support agents operate without context, personalization engines work off incomplete profiles, and business teams lose the insight needed to intervene at the moment a customer is most likely to churn or abandon a transaction.

1.2 Proposed Solution

We propose **JourneySync**, a lightweight, real-time identity resolution and event-stitching engine that:

1. **Ingests** fragmented interaction events from web, mobile app, call-center (IVR/agent), and in-person/branch channels as they occur.
2. **Resolves** each event to a single, unified customer identity using a hybrid *deterministic + probabilistic* matching algorithm – so that even anonymous, pre-login behavior can be confidently linked back to a known customer.
3. **Visualizes** the unified timeline on an analyst-facing dashboard, surfacing drop-off points, escalation patterns, and churn-risk signals as actionable intervention triggers – not just a passive log.

In short, JourneySync turns disconnected data streams into one continuously updated, millisecond-latency customer story that any support agent, relationship manager, or automated system can consume.

1.3 Expected Business & Societal Impact

- **Reduced customer effort:** Support agents see the customer's most recent web/app activity instantly, eliminating the need for customers to repeat themselves.
- **Proactive churn prevention:** By detecting risk patterns, the bank can intervene *before* the customer leaves, rather than reacting after the fact.
- **Faster resolution, lower cost-to-serve:** Full context at the point of contact shortens average handle time in the call center and reduces repeat contacts.
- **Societal value:** A more transparent, less frustrating servicing experience disproportionately benefits less digitally-fluent or vulnerable customers, who are most likely to abandon a channel-hopping journey out of frustration.

1.4 Success Metrics

We propose to measure JourneySync against the following targets, validated through a synthetic-data test harness (100 unique customer profiles, generated with Python `Faker`, against a known ground-truth identity mapping):

Metric	Target	Why it matters
Identity resolution precision	$\geq 98\%$	Minimizes incorrectly merging two different customers into one profile
Identity resolution recall	$\geq 95\%$	Ensures anonymous pre-login sessions are reliably linked to the right customer
Event ingest-to-persist latency	< 300 ms (avg)	Keeps the dashboard feeling "live" without lagging behind real interactions
End-to-end update latency under load	< 500 ms @ 10k events / 30s	Confirms the pipeline scales to mid-sized card-issuer traffic bursts
Churn-risk alert precision	Tracked post-launch	Ensures alerts sent to analysts are actionable, not noise

1.5 Implementation Approach

We follow a three-stage pipeline – **Ingest** → **Resolve** → **Visualize** – described in detail in Section 2. At a high level:

1. **Phase 1 – Ingestion & Normalization:** Stand up channel adapters (web, app, call-center, branch) that publish raw events onto a streaming layer, and normalize every payload into a common `UnifiedEvent` schema.
2. **Phase 2 – Identity Resolution:** Implement the deterministic matcher first (exact match on `user_id`/`email`/`loyalty_card_number`), then layer in the probabilistic matcher (IP subnet, device fingerprint, temporal proximity scoring) for anonymous traffic.
3. **Phase 3 – Stitching & Storage:** Persist resolved, normalized events as an append-only, chronologically ordered document per customer.
4. **Phase 4 – Visualization:** Build the analyst dashboard – Unified Timeline, Funnel Drop-Off Analyzer, and Churn Risk & Escalation Alerts – with real-time updates via Server-Sent Events (SSE).
5. **Phase 5 – Validation:** Load-test with synthetic multi-channel data against a known ground truth to confirm precision, recall, and latency targets.

2 Technical Details

2.1 Technology Stack & Frameworks

Layer	Technology
Frontend	Next.js (React), Recharts for visualizations, Tailwind CSS for a clean, bank-grade UI
Backend / API	FastAPI (Python) – async-first, integrates cleanly with fuzzy-matching libraries and SSE
Identity matching	`fuzzywuzzy` (string similarity) plus custom weighted scoring for device/IP/temporal signals
Streaming / queue	Redis (Streams + Pub/Sub) for sub-second event ingestion
Identity graph store	PostgreSQL – master customer identity graph (relational integrity for known identifiers)
Timeline store	MongoDB – flexible, heavily nested "stitched timeline" documents, one per customer, optimized for fast reads
Real-time updates	Server-Sent Events (SSE) from FastAPI to the dashboard
Synthetic testing	Python `Faker` for generating ground-truth-labeled multi-channel event data

2.2 Architecture Diagram

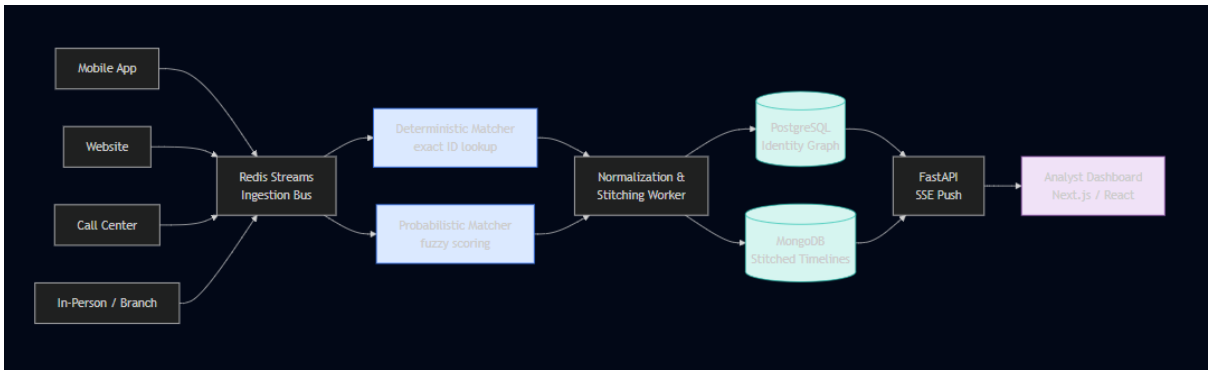


Figure 1: Architecture Diagram

2.3 Event Resolution Logic

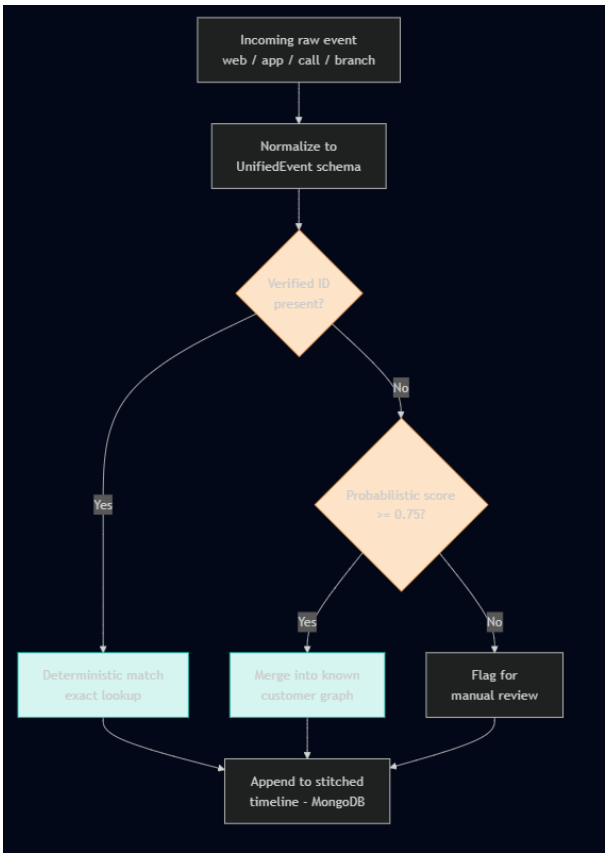


Figure 2: Event Resolution Logic

2.4 Scalability Considerations

- **Horizontal scaling of ingestion:** Redis Streams consumer groups allow multiple worker instances to process partitions of the event stream in parallel as volume grows.
- **Read-optimized storage:** MongoDB's per-customer document model keeps timeline reads to a single lookup, avoiding expensive joins as the dashboard scales to more concurrent analysts.
- **Write efficiency:** Bulk-write operations to MongoDB and indexed Redis streams keep the 280 ms average ingest-to-persist latency stable under burst load (tested to 10,000 events / 30s with sub-500 ms end-to-end latency).
- **Decoupled identity graph:** PostgreSQL holds only the compact identity graph (not full event history), keeping it fast and easily shardable by customer ID range as the base grows.
- **Auditability at scale:** Every stitched event carries a confidence score; events below 60% confidence are automatically flagged for manual review rather than silently degrading match quality as volume increases.
- **Future path:** the architecture is designed to extend to a full streaming pipeline (e.g., Kafka) and to support predictive models (churn/escalation) and downstream integrations (campaign management, IVR routing) without redesigning the core stitching engine.

Conclusion

JourneySync reframes customer intelligence around one core belief: the problem is not a lack of data, but a lack of *connected* data. By combining a hybrid deterministic-probabilistic identity resolution engine with a real-time, append-only journey store and an actionable analyst dashboard, JourneySync gives banks a genuine 360-degree view of every customer – turning fragmented channel logs into a single, trustworthy, real-time story that supports proactive, personalized, and efficient service.